

תכנון תחת אי-ודאות חלקית במיקום הסוכן — POMCP ו-POMDP בסביבת Box Pushing

רקע ומטרת המטלה

במטלות הקודמות פתרתם את בעיית ה-Box Pushing תחת הנחת תצפיתיות מלאה (full observability): בכל רגע נתון ידעתם בוודאות את מצב העולם המלא, כולל מיקומכם. במטלה זו תוסיפו לבעיה שכבת אי-ודאות חדשה: הסוכן (הרובוט) אינו יודע את מיקומו ההתחלתי בלוח, ועליו להעריך את מיקומו תוך כדי פעולה, אך ורק על סמך תצפיות חלקיות שהוא אוסף לאורך הדרך.

בעיה כזו מוגדרת כ-POMDP — Partially Observable Markov Decision Process. עליכם לפתור אותה באמצעות אלגוריתם POMCP — Partially Observable Monte-Carlo Planning — אלגוריתם תכנון מקוון (online) המבוסס על חיפוש עץ מונטה-קרלו (MCTS) הפועל ישירות מעל ייצוג חלקיקים (particles) של מצב האמונה, ללא צורך בייצוג הסתברותי מפורש ומלא של כל המצבים.

הגדרת ה-POMDP

מרחב המצבים (States)

מרחב המצבים זהה במהותו למרחב המצבים שהכרתם ממטלות קודמות, אך כעת מיקומו של הסוכן עצמו אינו ידוע. כלומר, עבור כל סוכן (רובוט) בנפרד, מרחב המצבים האפשריים למיקומו הוא אוסף כל המשבצות הפנויות בלוח — מספר המצבים האפשריים שווה למספר המשבצות הפנויות בלוח.

מיקומי הקופסאות (boxes) ושאר רכיבי העולם נשארים ידועים במלואם וקבועים; אי-הוודאות מוגבלת אך ורק למיקום הסוכן עצמו.

פעולות (Actions) ופונקציית המעבר (Transition Function)

הגדרת הפעולות זהה להגדרה ממטלה 1. הדינמיקה ההסתברותית זהה לזו שהוגדרה במטלה 2:

- פעולת תזוזה (move): מצליחה בהסתברות 0.8 בכיוון המבוקש; בהסתברות 0.1 מתבצעת סטייה לכיוון אחד, ובהסתברות 0.1 סטייה לכיוון השני.
- פעולת דחיפה (push): מצליחה בהסתברות 0.8; בהסתברות 0.2 הפעולה נכשלת ללא שינוי מצב.
- אם תנאי הקדם של הפעולה אינם מתקיימים (למשל, תזוזה אל קיר) — הפעולה אינה משנה את מצב העולם.

פונקציית התצפית (Observation Function)

בכל צעד הסוכן מקבל תצפית חלקית בלבד — "חלון" בגודל 3×3 מהלוח — במקום לדעת את מיקומו המדויק. עליכם לממש אחת משתי החלופות הבאות, לבחור אחת מהן ולנמק את הבחירה בדוח:

חלופה א' — תצפית כיוונית קבועה:

התצפית היא תמיד חלון 3×3 הממוקם צפונית למיקום הסוכן בפועל, ללא תלות בכיוון שאליו "פונה" הסוכן.

חלופה ב' — תצפית אגוצנטרית (ממוקדת בסוכן):

התצפית היא חלון 3×3 שבו הסוכן ממוקם בתא המרכזי.

בשני המקרים תוכן התצפית (אילו תאים פנויים, אילו קירות, והאם יש קופסה) הוא דטרמיניסטי ביחס למיקום האמיתי של הסוכן: $O(o | s', a) = 1$ אם ורק אם s הוא בדיוק החלון המתאים סביב s' , אחרת $O(o | s', a) = 0$. כלומר אין רעש חיישנים — מקור אי-הוודאות היחיד הוא חוסר הידיעה על המיקום ההתחלתי, יחד עם הדינמיקה ההסתברותית של הפעולות עצמן.

הערה חשובה: התצפית ה-"מובנית" (default) של MiniGrid אינה זהה לאף אחת משתי החלופות שתוארו לעיל. ברירת המחדל של MiniGrid היא חלון שפונה תמיד קדימה ביחס לכיוון שאליו "מביט" הסוכן (forward-facing, תלוי-כיוון, בגודל 7×7 כברירת מחדל) — לא חלון בכיוון מצפן קבוע, וגם לא חלון סימטרי הממוקד בסוכן. מכיוון שבסביבת Box Pushing שלנו פעולות התזוזה הן ישירות (שמאלה/ימינה/למעלה/למטה, ללא מושג של "כיוון פניה" לסוכן), לא ניתן לקבל אף אחת משתי החלופות "בחינם" דרך wrapper מובנה של MiniGrid כמו `ViewSizeWrapper`. יש לממש את פונקציית התצפית כפונקציה ייעודית משלכם, שמבצעת חיתוך (slice) ישיר מתוך ייצוג הלוח המלא הידוע למערכת (כפי שכבר חשוף, למשל, דרך הפונקציה שמייצרת את קובצי ה-PDDL) סביב מיקומו האמיתי — אך הנסתר מהסוכן עצמו — של הרובוט. ודאו מול המימוש בפועל של הסביבה שלכם אילו רכיבי MiniGrid, אם בכלל, רלוונטיים אצלכם.

פרס (Reward), תנאי סיום וגורם ההיוון

זהים להגדרה במטלות הקודמות (כולל γ).

מה עליכם לממש

1. פונקציית תצפית

לפי אחת משתי החלופות שפורטו לעיל.

2. מסנן חלקיקים (Particle Filter) למצב האמונה

- ייצוג: אוסף של N חלקיקים, כאשר כל חלקיק מייצג השערת מיקום אפשרית של הסוכן.
- אתחול: בתחילת הריצה, כשהסוכן אינו יודע את מיקומו, יש לפזר את החלקיקים באופן אחיד על פני כל המשבצות הפנויות בלוח.
- עדכון מצב האמונה (לאחר ביצוע פעולה אמיתית a וקבלת תצפית אמיתית o מהסביבה): בהתאם לאלגוריתם POMCP המקורי (Silver & Veness, 2010), זהו עדכון מבוסס דגימת-דחייה (rejection sampling) שאינו משוקלל: דוגמים חלקיק ממסנן החלקיקים הנוכחי, מדמים עליו את הפעולה a באמצעות אותו מודל גנרטיבי שמשמש את POMCP לתכנון (transition + observation), ובודקים אם התצפית המדומה שהתקבלה שווה לתצפית האמיתית o . אם כן — שומרים את המצב המתקבל כחלקיק חדש; אם לא — דוחים אותו ודוגמים שוב. התהליך חוזר עד לאיסוף N חלקיקים חדשים.
- שימו לב: מכיוון שפונקציית התצפית שהגדרתם דטרמיניסטית, שיעור הדחייה עלול להיות גבוה, ובמקרי קצה מסנן החלקיקים עלול "להתרוקן" לחלוטין (אם אף חלקיק לא הניב תצפית מתאימה). יש לטפל במקרה הזה במימוש — לדוגמה ע"י הגבלת מספר ניסיונות הדגימה והשלמה בעזרת תזרוע מחדש (particle reinvigoration), או ניצול הידיעה המלאה על מפת הלוח כדי לבדוק ישירות אילו מצבים עדיין עקביים עם התצפית שהתקבלה.
- תוצאת התהליך: מצב אמונה (belief state) מעודכן — אותו מסנן חלקיקים משמש הן כייצוג מצב האמונה האמיתי בין צעדי הסביבה, והן כייצוג מצב האמונה הפנימי בכל צומת בעץ החיפוש של POMCP.

3. אלגוריתם POMCP

- יש לממש את האלגוריתם כך שירוך ישירות מעל מסנן החלקיקים (ולא מעל פילוג הסתברותי מפורש), בהתאם לגישה המקורית של Silver & Veness (2010): בכל צעד תכנון, דוגמים מצב התחלתי s ממסנן החלקיקים הנוכחי, ומתחילים ממנו סימולציה (simulate).
- כל עוד הסימולציה נמצאת בתוך עץ החיפוש הקיים (היסטוריית action-observation שכבר ביקרתם בה) — בוחרים את הפעולה הבאה לפי כלל UCB1/UCT, המאזן בין ניצול (פעולות עם ערך משוער גבוה) לחקירה (פעולות עם מעט ביקורים).
- ברגע שהסימולציה יוצאת מהעץ הקיים (היסטוריה חדשה שלא ביקרתם בה) — מוסיפים צומת חדש לעץ, ומריצים rollout (לרוב באמצעות מדיניות אקראית או היוריסטית פשוטה) עד לעומק חיפוש מוגדר מראש (horizon), כדי להעריך את הערך מאותה נקודה.
- הערך המתקבל מסונכרן (back-propagated) במעלה העץ, ומעדכן את הסטטיסטיקות (מספר ביקורים, ערך משוער) של כל הצמתים שהסימולציה עברה בהם.
- לאחר מיצוי תקציב הזמן שהוקצב להחלטה, נבחרת בפועל הפעולה האמיתית בעלת הערך המשוער הגבוה ביותר בשורש העץ. לאחר ביצועה וקבלת תצפית אמיתית, יש לעדכן את מצב האמונה כמתואר בסעיף הקודם, ולעבור לצעד התכנון הבא.
- אין להשתמש בספריות RL / Planning / POMDP מוכנות (כגון pomdp_py) — יש לממש את POMCP, מסנן החלקיקים ופונקציית התצפית בעצמכם.

לולאת ה-Online Planning — מבנה כללי

הלולאה הכללית מחליפה את לולאת התכנון הקלאסי שהכרתם ממטלות קודמות (שם הופעלה תכנית PDDL בכל צעד) בלולאת תכנון מבוססת POMCP, הפועלת מעל מצב אמונה במקום מעל מצב ידוע:

```
obs, info = env.reset()
particles = init_uniform_particles(free_cells)

done = False
while not done:
    # 1. Plan: run POMCP over the current belief state
    action = run_pomcp(particles, time_budget=time_limit)

    # 2. Execute the real action in the environment, get a real observation
    obs, reward, terminated, truncated, info = env.step(action)

    # 3. Update the belief state given the action and the observation received
    particles = update_particles(particles, action, obs)

done = terminated or truncated
```

הרצת הניסוי

יש להריץ את האלגוריתם על הבעיה תחת שתי מגבלות זמן חישוב שונות, לכל החלטה בודדת: שנייה אחת (1s), ו-20 שניות (20s). המימוש שלכם חייב להבטיח שההחלטה אכן מתקבלת בתוך מגבלת הזמן שנקבעה — כלומר, יש לאכוף את ה-budget בפועל בתוך עץ החיפוש של POMCP, ולא רק לציין אותו כהערה.

עבור כל אחת משתי מגבלות הזמן, הריצו את האלגוריתם בסימולטור 30 פעמים, ודווחו את מספר הצעדים הממוצע וסטיית התקן הנדרשים לפתרון המשימה.

פרמטרים מומלצים כנקודת מוצא

פרמטר	ערך מומלץ / הערה
time_budget_short	1 שניה
time_budget_long	20 שניות
n_runs	30
n_particles	יש לבחור ולנמק; מומלץ להתחיל מסביב ל-500, ולהגדיל אם מסתמן דלדול חלקיקים
horizon / max_depth (עומק חיפוש ל-rollout)	יש לקבוע בהתאם לגודל הלוח ול-gamma; פרמטר נדרש למימוש תקין של POMCP
UCB1 exploration constant (c)	יש לכוון; מומלץ להתחיל מ- $c \approx 1$
gamma	כפי שהוגדר במטלות הקודמות

הגדרת סביבת העבודה

סביבת העבודה זהה לזו ששימשה במטלות הקודמות (Box Pushing), ומיישמת כעת גרסת POMDP של אותה בעיה. אם טרם התקנתם את הסביבה, ראו הוראות התקנה מלאות במטלה 1 או בקובץ ה-README.md שב-repository:

<https://github.com/ronberg-bgu/RL-course>

יצירת Branch למטלה זו

```
git checkout -b student-{firstname}-ex4
```

לדוגמה: student-yossi-ex4, student-sarah-ex4

הגשה

בסיום, דחפו את ה-branch ופתחו Pull Request ב-repository של הקורס.

מה יש להגיש

- קוד מלא של: פונקציית התצפית, מסנן החלקיקים, ואלגוריתם POMCP.
- טבלת תוצאות: מספר הצעדים הממוצע וסטיית התקן, עבור כל אחת משתי מגבלות הזמן (1 שניה, 20 שניות), עבור תרחיש הסוכן הבודד ועבור תרחיש שני הרובוטים.
- פירוט ההיפר-פרמטרים שבהם השתמשתם (מספר חלקיקים, פרמטר חקירה ב-UCB1, עומק/אורך rollout, וכל פרמטר נוסף).
- הסבר קצר על בחירת חלופת התצפית (א' או ב') ועל הנימוק לבחירה.
- דיון קצר בתוצאות: כיצד מגבלת זמן החישוב (1 שניה מול 20 שניות) השפיעה על איכות ההחלטות ועל מספר הצעדים הממוצע? כיצד השפיע התרחיש הרב-סוכני (שני רובוטים) על הביצועים, בהשוואה לסוכן בודד?

הערות חשובות

- אין להשתמש בספריות RL / Planning / POMDP מוכנות (כגון pomdp_py כפתרון מן המוכן) — יש לממש את POMCP, מסנן החלקיקים ופונקציית התצפית בעצמכם.
- יש לוודא שמגבלת זמן החישוב (1 שניה / 20 שניות) אכן נאכפת בפועל בקוד (budget לעץ החיפוש של POMCP), ולא רק מצוינת כהערה.
- אי-הוודאות במטלה זו נוגעת אך ורק למיקום הסוכן עצמו; מיקומי הקופסאות ושאר רכיבי העולם ידועים במלואם.
- יש לדווח בבירור על כל שינוי מהפרמטרים המומלצים, ככל שבוצע.